

Lab 3: Kafka Core Concepts and Terminology

Lab 3.1: Building a Kafka Producer-Consumer Pipeline

Goal:

Implement a basic Kafka pipeline that includes a producer sending messages to a topic and a consumer reading those messages. You will also explore consumer groups and message offsets.

Instructions

Step 1: Set Up Kafka Environment

1. Ensure Kafka is running locally or on a server.
2. Start Zookeeper and Kafka broker using the default configuration:

```
bin/zookeeper-server-start.sh config/zookeeper.properties
bin/kafka-server-start.sh config/server.properties
```

Step 2: Create a Kafka Topic

Run the following command to create a topic named `lab-topic` with 3 partitions and a replication factor of 1:

```
bin/kafka-topics.sh --create --topic lab-topic --bootstrap-server
localhost:9092 --partitions 3 --replication-factor 1
```

Step 3: Implement the Producer

1. Create a file named `LabProducer.java`.
2. Paste the following code into the file:

LabProducer.java

```
import org.apache.kafka.clients.producer.*; import
org.apache.kafka.common.serialization.StringSerializer; import java.util.Properties;

public class LabProducer { public static void main(String[] args) { Properties props = new
Properties(); props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"localhost:9092"); props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

    Producer<String, String> producer = new KafkaProducer<>(props);

    for (int i = 0; i < 10; i++) {
        ProducerRecord<String, String> record = new ProducerRecord<>("lab-
topic", "Message " + i);
```

```

        producer.send(record);
        System.out.println("Sent: Message " + i);
    }

    producer.close();
}
}

```

Step 4: Implement the Consumer

1. Create a file named `LabConsumer.java`.
2. Paste the following code:

`LabConsumer.java`

```

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class LabConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "lab-consumer-group");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

        Consumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("lab-topic"));

        while (true) {
            ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
            for (ConsumerRecord<String, String> record : records) {
                System.out.printf("Consumed: key=%s, value=%s, offset=%d%n",
                    record.key(), record.value(), record.offset());
            }
        }
    }
}

```

Step 5: Compile and Run the Applications

1. Compile both Java files:

```
javac -cp $(echo /path/to/kafka/libs/*.jar | tr ' ' ':')  
LabProducer.java LabConsumer.java
```

2. Run the producer:

```
java -cp .:$(echo /path/to/kafka/libs/*.jar | tr ' ' ':') LabProducer
```

3. In a separate terminal, run the consumer:

```
java -cp .:$(echo /path/to/kafka/libs/*.jar | tr ' ' ':') LabConsumer
```

Step 6: Observe the Output

The consumer should output something like:

```
Consumed: key=null, value=Message 0, offset=0  
Consumed: key=null, value=Message 1, offset=1  
...  
Consumed: key=null, value=Message 9, offset=9
```

4. Complete Implementation

Final Files

LabProducer.java

```
import org.apache.kafka.clients.producer.*;  
import org.apache.kafka.common.serialization.StringSerializer;  
import java.util.Properties;  
  
public class LabProducer {  
    public static void main(String[] args) {  
        Properties props = new Properties();  
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");  
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,  
StringSerializer.class.getName());  
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,  
StringSerializer.class.getName());  
  
        Producer<String, String> producer = new KafkaProducer<>(props);  
  
        for (int i = 0; i < 10; i++) {  
            ProducerRecord<String, String> record = new  
ProducerRecord<>("lab-topic", "Message " + i);  
            producer.send(record);  
            System.out.println("Sent: Message " + i);  
        }  
    }  
}
```

```

    }

    producer.close();
}
}

```

 LabConsumer.java

```

import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.serialization.StringDeserializer;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class LabConsumer {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ConsumerConfig.GROUP_ID_CONFIG, "lab-consumer-group");
        props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class.getName());
        props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");

        Consumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList("lab-topic"));

        while (true) {
            ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
            for (ConsumerRecord<String, String> record : records) {
                System.out.printf("Consumed: key=%s, value=%s, offset=%d%n",
                    record.key(), record.value(), record.offset());
            }
        }
    }
}

```

Summary

In this chapter, you explored Kafka's core architecture and components, including topics, partitions, brokers, producers, and consumers. You also learned how offsets and replication ensure message ordering and fault tolerance. Through the hands-on lab, you built a basic Kafka producer-consumer pipeline and observed how messages are sent and consumed in a distributed environment.